Checked Norm and Metric Arithmetic in Lean: A Finite Core and a Library Evaluation

Anonymous Authors

Abstract

A norm estimate can use a linear decomposition that the goal does not show. We give a finite reduction for affine norm comparisons over real seminormed spaces, with rational vector terms and vector-equality assumptions. The reduction includes strict comparisons and distances. Its finite completeness theorem does not remove implementation resource limits. Lean validates the vector identities and norm certificates. Bounded extensions specialize local quantified bounds and check finite metric constraints.

An earlier census processes 8637 tracked mathlib files. A targeted follow-up processes 885 files and records 10 incomplete outcomes. The verified local patch replaces 87 proof spans and removes 310 lines after preparation, helpers, and imports. With built dependencies, the ratio of mean original and replacement compilation times is 1.0724 across 46 affected files. The conditional paired 95% interval is $[1.0669, 1.0814]$.

1 The problem and the trust boundary

A proof can use many lines to specify a norm decomposition. For example, two supplied bounds imply

$$\|x + y\| \le a, \quad \|x - y\| \le b \quad \Longrightarrow \quad \|x\| \le (a + b)/2.$$

The hidden identity is $x = \frac{1}{2}(x+y) + \frac{1}{2}(x-y)$. The tactic finds rational coefficients and constructs a proof from them. Here E is a real seminormed space.

```
example (x y : E) (a b : ℝ)
    (h₁ : ‖x + y‖ ≤ a) (h₂ : ‖x - y‖ ≤ b) :
    ‖x‖ ≤ (a + b) / 2 := by
  norm_arith
```

Lean's kernel validates the proof terms that tactics produce [de Moura and Ullrich, 2021]. Exact rational search proposes certificates for proved norm lemmas. The search algorithm stays outside the trusted kernel. The finite theorem, checked certificates, and bounded program have distinct guarantees. A program completeness claim must have a completed search.

Artifact status. The expanded tactic has six checked examples. The targeted follow-up records its incomplete outcomes. The historical full census has separate outcomes. The composed source patch passed combined verification. The new confirmatory run met its predeclared speed criterion.

2 The real-space fragment

A vector term is a rational linear combination of formal atoms $x_1, \ldots, x_d$. A scalar term is a rational affine combination of real scalar atoms $s_1, \ldots, s_k$ and norm terms $\|v\|$. Assumptions are vector equalities and scalar comparisons with $\le$, $<$, or $=$. Conclusions use the same scalar-comparison

syntax. Validity means truth in all such spaces for all assignments of vector and scalar atoms that satisfy the recognized assumptions.

Distances abbreviate $\|u - v\|$. Products of norms are not in this affine scalar grammar.

Norm-free scalar expressions and unsupported vector subterms can be independent atoms. This includes vector terms with variable coefficients. Recognized equalities can relate vector atoms. Completeness applies to the reified abstraction, without the internal semantics of opaque expressions.

For each vector equality $u = v$, include the norm argument $u - v$ with value zero. Let $v_1, \dots, v_m \in \mathbb{Q}^d$ be all observed norm arguments. Let Z index the differences introduced from vector equalities. Write $A = [v_1 \ \cdots \ v_m]$ and $n_i = \|v_i\|$. Define the pointed polyhedron

$$P_i = \{z \in \mathbb{R}^{2m}_{\geq 0} : [A, -A]z = v_i\}. \tag{1}$$

Its vertices are a finite rational set B_i. The finite norm constraints are

$$n_i \geq 0, \quad n_\ell = 0 \ (\ell \in Z), \qquad n_i \leq \sum_{j=1}^{m} (z_j + z_{m+j}) n_j \quad (i \leq m, \ z \in B_i). \tag{2}$$

One validated vector identity, the triangle inequality, and real homogeneity prove each inequality.

Theorem 1 (Finite norm-value characterization)**.** *Nonnegative real values $n_1, \dots, n_m$ satisfy* (2) *if and only if some real seminormed space realizes these values and the vector equalities. Thus exact linear arithmetic over these constraints decides each implication in the declared fragment.*

3 Finite search and checked certificates

A vertex of P_i has independent positive-support columns. Such a support contains at most $\operatorname{rank}(A)$ columns. It cannot contain the two signs of one nonzero column. Thus independent subsets of the unsigned source columns suffice: solve each selected system exactly and keep its signed coefficients. The zero target also has the empty certificate.

This enumeration is finite, although it can have exponential cost. An implementation must report resource exhaustion as unknown.

For a certificate $v = \sum_j c_j v_j$, Lean constructs

$$\|v\| \leq \sum_j |c_j| \, \|v_j\| \, . \tag{3}$$

Normalization proves the vector identity. Norm lemmas prove the inequality. The scalar backend combines these facts with local scalar assumptions.

For equality conclusions, prove the two directions. For a strict conclusion, the backend adds its weak negation. An equality's negation has two strict branches. The reference reduction must have exact feasibility for mixed linear constraints. It does not use a numerical tolerance.

Completeness argument. For fixed feasible n, assign a cost to each signed representation:

$$p(u) = \min\{\textstyle\sum_j n_j (z_j + z_{m+j}) : [A, -A]z = u, \ z \geq 0\}. \tag{4}$$

This minimum exists on the span of the observed vectors. A vertex attains a minimum because the feasible polyhedron is pointed and the cost is nonnegative. The constraints give $p(v_i) \geq n_i$. The single-column representation gives the reverse inequality.

The representation costs define a seminorm. Extend it by zero on a complement and quotient by its zero subspace. The quotient satisfies the encoded vector equalities and preserves observed norm values. Thus a scalar counterexample gives a space counterexample to the abstraction. Appendix A gives the cases with zero costs and vector relations.

One certificate interface. The default `norm_arith` selects related local bounds automatically. The explicit form `norm_arith [facts]` gives the caller control. The two paths use the same normal forms and certificate layer. Fast paths can return before the reference search. The finite reference search is the fallback.

Bounded extensions. The tactic projects conjunctions and specializes local quantified norm bounds at observed vector terms. It checks local, conjunction, and linear scalar premises. Explicit fact lists exclude omitted norm and distance premises. This does not give quantified completeness.

A separate path gives nonnegativity, symmetry, self-distance, and triangle constraints for at most eight observed points in pseudometric spaces. Failed metric attempts restore state before the norm paths run. This path does not use vector-space structure. The additive-group extension uses integer certificates without real homogeneity. Appendix B gives checked examples and the fixed limits.

4 Library evaluation

The mathlib proofs are the baseline [The mathlib Community, 2020]. The initial census uses all tracked Lean files in `Mathlib/` at one pinned revision. It records source-backed tactic and term observations during elaboration [The Lean Development Team, 2026]. Each file has a completion status. The denominator includes incomplete extraction.

The follow-up uses the fixed subset with norm observations. Its outcomes are separate from the earlier full census outcomes. The composed patch combines census-generated replacements with replacements composed outside the census.

The historical full census processed 8637 files, with 27 incomplete outcomes. The targeted follow-up processed 885 files. 10 are incomplete. It does not show new coverage of the full universe. The composed patch replaces 87 spans in 46 files. It removes 310 code lines after all preparation and imports.

Eligibility uses the recognized norm grammar and an editable source span. The follow-up includes the real-space core, additive groups, the opaque upper-bound extension, and generic metric comparisons. The scan records the original goal before each tactic attempt. It uses fixed preparation and resource rules. The scan does not try original single-line proofs because a replacement cannot use fewer proof lines. These untried observations are not tactic successes.

The deterministic selector excludes overlapping replacements and files without positive savings after import costs. All selected replacements compile together with the theorem statements and assumptions unchanged. The selected proof lines satisfy mathlib's 100-character limit. The proof counts include preparation, helpers, and imports. Tactic implementation and census infrastructure have separate costs in Appendix C. These are local patch savings, not a reduction in total repository size.

A selected source replacement. The entry `manual-strong-law-sums` replaces one estimate inside the Strong Law proof. It keeps the finite sum bound and its pointwise normalization. The tactic then combines the centered average and expectation bounds.

```
simp only [Pi.sub_apply] at hn
have hsum : ‖(n : ℝ)⁻¹ • ∑ i ∈ range n, (X i ω - Y k i ω)‖ ≤ δ := by
  grw [norm_smul, norm_inv, RCLike.norm_natCast, norm_sum_le, ← div_eq_inv_mul,
  hn.le]
simp only [sum_sub_distrib, smul_sub] at hsum
norm_arith
```

The block decreases from 20 proof code lines to 5, with all preparation counted. The net figure includes the import one time for each file. Appendix B records the original block, declaration, and provenance.

Compilation results. The statistic was the ratio of mean total times for all affected files. We froze the code, patch, sample count, and analysis before the confirmatory run. The predeclared speed criterion was a paired 95% interval fully above one. The run included tactic search, imports, proof construction, proof validation, serialization, and C generation. It excluded native C object compilation and Lake dispatch. The analysis kept slower files and all paired samples.

Appendix C gives the timing-block method and runtime limits.

Mean original and replacement full-pass times were 131.188 and 122.333 seconds. The ratio of mean full-pass times is 1.0724. The conditional paired 95% interval is [1.0669, 1.0814]. The predeclared speed criterion is met. Compilation was slower for 7 files. The appendix reports every file.

The variants used different native imports. This ratio does not give an algorithm comparison. The result applies to these affected files with built dependencies on the recorded machine.

5 Related work and limits

HOL Light has `NORM_ARITH`, and Isabelle has the `norm` method [Harrison, 2026, Chaieb, 2026]. Solovay, Arthan, and Harrison prove decidability results for universal normed-space formulas [Solovay et al., 2012]. We give a Lean certificate design, an explicit finite reduction, and a reproducible library evaluation. We do not give the first decision procedure.

The finite theorem gives no parser coverage result, complete census, or practical runtime bound. The corpus runs include incomplete extraction and untried observations. The targeted follow-up does not show all uses in mathlib. Proof compression does not prove faster compilation.

References

Aerospace, Security and Defence Industries Association of Europe. *ASD-STE100 Simplified Technical English: Standard for Technical Documentation*, issue 9 edition, 2025. URL https://www.asd-ste100.org/assets/files/ASD-STE100_ISSUE9.pdf.

Amine Chaieb. Isabelle/HOL: Norm_Arith and the norm arithmetic procedure. Source revision e29bb4ce04cdb891585ff18b1750b8ec8c805b4c, 2026. URL https://github.com/isabelle-prover/mirror-isabelle/blob/e29bb4ce04cdb891585ff18b1750b8ec8c805b4c/src/HOL/Analysis/normarith.ML. Accessed 20 October 2026.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635, 2021. doi: 10.1007/978-3-030-79876-5_37. URL https://lean-lang.org/papers/lean4.pdf.

Thomas J. DiCiccio and Bradley Efron. Bootstrap confidence intervals. *Statistical Science*, 11 (3):189–228, 1996. doi: 10.1214/ss/1032280214. URL https://doi.org/10.1214/ss/1032280214. With discussion.

Bradley Efron. Better bootstrap confidence intervals. *Journal of the American Statistical Association*, 82(397):171–185, 1987. doi: 10.1080/01621459.1987.10478410. URL https://doi.org/10.1080/01621459.1987.10478410.

C. A. Field and A. H. Welsh. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(3):369–390, 2007. doi: 10.1111/j.1467-9868.2007.00593.x. URL https://doi.org/10.1111/j.1467-9868.2007.00593.x.

John Harrison. HOL Light: NORM_ARITH in Multivariate/vectors.ml, 2026. URL https://github.com/jrh13/hol-light/blob/master/Multivariate/vectors.ml. Accessed 20 October 2026.

Robert M. Solovay, R. D. Arthan, and John Harrison. Some new results on decidability for elementary algebra and geometry, 2012. URL https://arxiv.org/abs/0904.3482v3.

The Lean Development Team. Lean 4 information trees. Source at Lean v4.36.0-rc1, 2026. URL https://github.com/leanprover/lean4/blob/v4.36.0-rc1/src/Lean/Elab/InfoTree/Types.lean. Accessed 20 October 2026.

The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 2020. doi: 10.1145/3372885.3373824. URL https://doi.org/10.1145/3372885.3373824.

A Full proof of the finite characterization

Validity here quantifies over all real seminormed spaces and all assignments of vector and scalar atoms. Formal atoms can be linearly dependent in an input. The reduction records all recognized rational vector-equality assumptions.

Scalar atoms and unsupported vector subterms have independent interpretations in the abstraction. A countermodel can ignore the internal operations of opaque expressions. This is completeness for the abstraction. Lean validates each proof for the concrete goal.

A.1 Encoding vector equalities

Represent the original atoms by the standard basis of $\mathbb{R}^d$. Set $V = \mathbb{R}^d$ and represent each observed term by its rational coefficient vector. For each recognized vector equality $u = v$, add the observed difference $e = u - v$ and the scalar constraint $n_e = 0$. In the original context, the equality proves this zero-norm fact in Lean.

A nonzero vector can have zero norm in an arbitrary seminormed space. The necessary converse is only a model construction for scalar conclusions. First realize all norm values on V. Then construct the quotient by the zero subspace of that seminorm. Each encoded difference then becomes zero. The quotient preserves each observed norm value.

Thus the scalar encoding is exact for the stated implication problem. This justifies the frontend's zero-norm encoding. The frontend does not construct an explicit vector-space quotient.

Let $U \subseteq V$ be the span of the observed vectors $v_1, \ldots, v_m$. All constructions start on U. The construction includes repeated vectors, opposite vectors, and zero vectors.

Lemma 1 (Vertices and finite enumeration)**.** *Let M be a rational matrix and let b be real. For $P = \{z \geq 0 : Mz = b\}$, a feasible point is a vertex exactly when its positive-support columns are independent. There are a finite number of vertices. If b is rational, all vertices are rational. If P is nonempty and a nonnegative linear cost is bounded below on it, a minimum occurs at a vertex.*

Proof. Suppose the support columns of a feasible z are dependent. Select a nonzero vector h with support there and $Mh = 0$. For a sufficiently small $\epsilon > 0$, the points $z+\epsilon h$ and $z-\epsilon h$ are nonnegative. They are distinct feasible points with midpoint z. Thus z is not a vertex.

Suppose the support columns are independent. If $z = (a + b)/2$ with feasible a, b, each zero coordinate of z is zero in a and b. The difference $a - b$ thus has support on the independent columns. Because $M(a - b) = 0$, we get $a = b = z$. This proves the vertex characterization.

Each independent support determines at most one solution. There are a finite number of supports. Thus there are a finite number of vertices. When b is rational, exact rational elimination gives rational coordinates for each such solution.

For the existence of an optimal vertex, write the cost as $w^T z$ with $w \geq 0$. Let z be a feasible point with dependent support. Select h as above. If h has the two signs, select the direction with nonpositive cost change. Move to the first coordinate boundary in that direction. This preserves feasibility, does not increase the cost, and decreases the support.

If h has only one sign, select it nonnegative. Move in direction $-h$. Its cost change is nonpositive because $w \geq 0$. Again the move stops at a coordinate boundary.

Do this step again to get a feasible vertex with cost no greater than that of z. The finite vertex set is nonempty and has a least-cost element. Each feasible point has cost no less than the minimum. Thus the vertex attains the infimum over P. □

The proof's last claim applies to each nonnegative cost used here. Strictly positive norm values and a bounded feasible polyhedron are not necessary. For $M = [A, -A]$, the positive support cannot contain the two copies of a nonzero column. The signed solution on an independent unsigned subset thus represents the same vertex data. Zero columns add no positive coordinate at a vertex. The all-zero point is a vertex when $b = 0$.

Lemma 2 (Necessity)**.** *Each vector assignment that satisfies the recognized equalities has norm values that satisfy* (2)*.*

Proof. The assignment induces a seminorm on V through a linear map. Seminorms are nonnegative, and the encoded equality differences have norm zero. For $z \in P_i$, the equality $[A, -A]z = v_i$ gives

$$v_i = \sum_j z_j v_j + \sum_j z_{m+j}(-v_j).$$

Subadditivity, absolute homogeneity, and $z \geq 0$ give $\|v_i\| \leq \sum_j (z_j + z_{m+j}) \|v_j\|$. This holds for all feasible points. Thus it holds for each vertex. □

Lemma 3 (Realization)**.** *Suppose nonnegative $n_1, \ldots, n_m$ satisfy* (2)*. There is a seminorm p on V with $p(v_i) = n_i$ for each i.*

Proof. For $u \in U$, the feasible set in (4) is nonempty. A real linear representation of u in the columns of A splits into positive and negative parts. Its cost is nonnegative. Lemma 1 shows that the minimum defining $p(u)$ exists and is finite. The zero representation gives $p(0) = 0$.

If z represents u, an exchange of its first and last m coordinates represents $-u$ with equal cost. Thus $p(-u) = p(u)$. For $\lambda > 0$, multiplication by λ gives a bijection between the representations of u and λu. Costs multiply by λ. Thus $p(\lambda u) = \lambda p(u)$. Symmetry and the zero case give $p(\lambda u) = |\lambda| p(u)$ for each real λ.

If z and z' minimize the costs of u and u', then $z+z'$ represents $u+u'$. Thus $p(u+u') \leq p(u)+p(u')$. These facts prove that p is a seminorm on U.

The representation of v_i by one positive column has cost n_i. Thus $p(v_i) \leq n_i$. By Lemma 1, an optimal representation of v_i is a vertex of P_i. Its constraint in (2) gives $n_i \leq p(v_i)$. Thus $p(v_i) = n_i$.

Select a real linear complement W with $V = U \oplus W$. Define $\widetilde{p}(u + w) = p(u)$ for $u \in U$ and $w \in W$. The uniqueness of the direct-sum decomposition makes this well-defined. Linearity of the projection onto U preserves homogeneity and subadditivity. Thus $\widetilde{p}$ is the stated seminorm on V. □

Proof of Theorem 1. Lemma 2 proves necessity. Each vector equality also gives its recorded zero-norm constraint. For sufficiency, apply Lemma 3 on $V = \mathbb{R}^d$. Let $K = \{v : p(v) = 0\}$ be the kernel of this seminorm. Homogeneity and subadditivity make K a vector subspace, and p induces a norm on V/K. Assign each formal atom its basis class in that quotient.

Each encoded equality difference belongs to K. Thus the assignment satisfies each vector equality. The quotient preserves $p(v_i) = n_i$ for each observed term. It is a normed space and thus also a seminormed space.

Replace each observed norm by a real variable n_i. Conjoin (2), the scalar assumptions, and the negation of the proposed conclusion. The resulting problem is rational linear feasibility for a finite system with strict and weak comparisons. A feasible solution extends to a seminormed-space counterexample by the realization lemma.

By necessity, a counterexample gives a feasible solution. Thus infeasibility is equivalent to validity of the implication. An equality conclusion is the conjunction of two weak inequalities, or equivalently its negation splits into two strict branches. Exact linear arithmetic decides each finite branch. This proves mathematical completeness of the reference reduction. □

The intermediate construction can have a nontrivial seminorm kernel. Its quotient is a normed-space model with the same scalar norm values and the necessary vector equalities. This observation is valid only for a grammar without vector-disequality assumptions or vector-equality conclusions. For multiple vector types, apply the construction to each type and combine their scalar constraints. Different spaces share scalar values but not vector identities.

A.2 Removing the target from the source set

The full polyhedron in (1) always contains the trivial single-column representation. An implementation can exclude v_i during the generation of nontrivial bounds on n_i. This optimization preserves the finite characterization.

Use a representation $v_i = c_i v_i + \sum_{j \neq i} c_j v_j$. If $|c_i| \geq 1$, nonnegativity implies its norm inequality. If $|c_i| < 1$, then $c_i \neq 1$ and

$$v_i = \sum_{j \neq i} \frac{c_j}{1 - c_i} v_j.$$

The target-free constraints bound n_i by the minimum cost of such representations. Lemma 1 reduces the minimum to their vertices. Because $|1 - c_i| \geq 1 - |c_i|$, this bound implies

$$(1 - |c_i|) n_i \leq \sum_{j \neq i} |c_j| n_j,$$

which is the original self-containing inequality.

If no target-free representation exists, no such $|c_i| < 1$ representation exists. The trivial inequalities add no restriction.

A.3 Strict scalar comparisons

Strict comparisons do not use approximate arithmetic. One exact feasibility construction replaces all strict rows $a_r^T y < b_r$ by $a_r^T y \leq b_r - \delta$, adds $0 \leq \delta \leq 1$, and maximizes δ. The original finite system is feasible exactly when the weak rows are feasible and this bounded linear program has a positive optimum.

For a feasible strict system, its finite set of positive gaps has a minimum that gives such a δ. The converse follows directly from $\delta > 0$. If there are no strict rows, weak feasibility suffices. This is one reference argument. It does not specify the Lean backend reduction.

B Checked examples and source provenance

The release records a successful source compilation for all marked blocks. This build checked their saved hashes. The generated manifest records source hashes and marker ranges. The paper does not change the imported blocks.

B.1 Quantified bounds and generic metric chains

The quantified example specializes the local hypothesis at two observed arguments. It normalizes ball membership, proves the radius premises with restricted scalar arithmetic, and checks the cancellation identity. It does not search arbitrary values of the quantified variable.

```
example (f : E → E) (x y z c : E) (r R a : ℝ)
    (h : ∀ t ∈ Metric.ball x R, ‖f t - c‖ ≤ a)
    (hy : y ∈ Metric.ball x r) (hz : z ∈ Metric.ball x r) (hr : r < R) :
    ‖f z - f y‖ ≤ a + a := by
  norm_arith
```

The metric example uses no addition or scalar multiplication on its points. Here X is a pseudometric space. Symmetry reverses the second bound. Triangle constraints and scalar arithmetic complete the chain.

```
example (a b c d : X) (r s t : ℝ)
    (h₁ : dist a b ≤ r) (h₂ : dist c b ≤ s) (h₃ : dist c d ≤ t) :
    dist a d ≤ r + s + t := by
  norm_arith
```

The specialization limits are 12 templates, 12 binders per template, 32 observed vectors, 32 new bounds, and 128 candidate attempts. Each attempt checks at most eight additive patterns of depth at most eight. Premise construction has depth four. Norm alias filtering has depth eight and a 256-node budget. When inspection is incomplete, the tactic excludes that expression.

The fixed rewrite set has rules for distances, balls, closed balls, the four bounded interval types, and membership in sets defined by predicates. This is not an unrestricted simplifier or quantified decision procedure. The metric path collects at most eight points in total. These limits give partial coverage, without changing Theorem 1.

B.2 Strict and reverse-triangle reasoning

The reverse-triangle estimate compares a difference of norms with the norm of a vector difference. The finite constraints supply a triangle bound. Scalar arithmetic rearranges it. The comparison grammar also includes strict scalar relations.

```
example (x y : E) : ‖x‖ - ‖y‖ ≤ ‖x - y‖ := by
  norm_arith
```

B.3 A checked boundary and the additive extension

Products of norms are not in the declared scalar grammar. This statement is true. In the negative tactic test, `norm_arith` must reject that unsupported goal. A multiplication lemma completes the Lean proof. This records an unsupported input. It is not a counterexample to the theorem or a timeout.

```
example (x : E) : 0 ≤ ‖x‖ * ‖x‖ := by
  fail_if_success norm_arith
  exact mul_self_nonneg _
```

For $n \in \mathbb{N}$, the additive groups give $\|n \bullet x\| \leq n \|x\|$. This does not give real homogeneity. For example, the discrete norm on $\mathbb{Z}$ is zero at zero and one at all nonzero inputs. At $x = 1$, $\|x\|$ and $\|2x\|$ are one. Thus $\|2x\| \leq 1$ does not imply $\|x\| \leq 1/2$ in that class. Do not use the real-space certificate in this class.

B.4 The selected composed patch example

The selected entry is `manual-strong-law-sums`. Its source is `Mathlib/Probability/StrongLaw.lean`, lines 760–780. Its declaration is `ProbabilityTheory.strong_law_ae_of_measurable`. The final combined source check includes this entry.

The source replacement changes one estimate inside the Strong Law argument. The preparation uses the existing finite sum bound, norm homogeneity, and the sum triangle inequality. It distributes subtraction and scalar multiplication before the tactic combines the bounds. It preserves the full theorem statement and its assumptions. The tactic does not prove the convergence hypotheses or the Strong Law.

Original source block.

```
  -- at such an `n`, the strong law is realized up to `ε`.
  calc
  ‖(n : ℝ)⁻¹ • ∑ i ∈ Finset.range n, X i ω - μ[X 0]‖
    = ‖(n : ℝ)⁻¹ • ∑ i ∈ Finset.range n, (X i ω - Y k i ω) +
        ((n : ℝ)⁻¹ • ∑ i ∈ Finset.range n, Y k i ω - μ[Y k 0]) + (μ[Y k 0] - μ[X
        0])‖ := by
      congr
      simp only [sum_sub_distrib, smul_sub]
      abel
  _ ≤ ‖(n : ℝ)⁻¹ • ∑ i ∈ Finset.range n, (X i ω - Y k i ω)‖ +
        ‖(n : ℝ)⁻¹ • ∑ i ∈ Finset.range n, Y k i ω - μ[Y k 0]‖ + ‖μ[Y k 0] - μ[X
        0]‖ :=
      norm_add₃_le
  _ ≤ (∑ i ∈ Finset.range n, ‖X i ω - Y k i ω‖) / n + δ + δ := by
      gcongr
      simp only [norm_smul, norm_inv, RCLike.norm_natCast,
        div_eq_inv_mul]
      gcongr
      exact norm_sum_le _ _
  _ ≤ δ + δ + δ := by
      gcongr
      exact hn.le
  _ < ε := hδ
```

B.5 An operator estimate

The source example is the candidate `fderiv-telescope` at the pinned mathlib revision. The build extracts the original and replacement blocks from the recorded manifest and pinned source. The generation code validates their source hashes. This checked example abstracts the shared bound and renames the four operator values. It generalizes the proof shape. It does not give the literal mathlib source.

Here E is an additive group.

```
example (w x y z : E) (b : ℝ)
    (J1 : ‖w - x‖ ≤ b) (J2 : ‖x - y‖ ≤ b) (J3 : ‖y - z‖ ≤ b) :
    ‖w - z‖ ≤ 3 * b := by
  norm_arith [J1, J2, J3]
```

Original source block.

```
    calc
      ‖L e p q - L e' p' q'‖ =
          ‖L e p q - L e p r + (L e p r - L e' p' r) + (L e' p' r - L e' p' q')‖
          := by
        congr 1; abel
      _ ≤ ‖L e p q - L e p r‖ + ‖L e p r - L e' p' r‖ + ‖L e' p' r - L e' p' q'‖
      :=
        norm_add₃_le
      _ ≤ 4 * ‖c‖ * (1 / 2) ^ e + 4 * ‖c‖ * (1 / 2) ^ e + 4 * ‖c‖ * (1 / 2) ^ e
      := by gcongr
      _ = 12 * ‖c‖ * (1 / 2) ^ e := by ring
```

Replacement source block.

```
    norm_arith [J1, J2, J3]
```

C Evaluation protocol and artifact status

The study uses Lean `4.36.0-rc1` and this mathlib revision:

```
2f3c7c8a8949b04a32f9303f59163acffb766996
```

Implementation paths. The frontend rewrites distances, collects norm arguments, and encodes recognized vector equalities as checked zero-norm facts. It tries structural bounds and short decompositions before the finite reference family. The reference search records the status of its independent-support enumeration: complete or resource-exhausted. Its default limits are 10,000 support systems and 10,000 retained results. The support-attempt count excludes the initial full-system rank test.

The public interface also keeps a sound upper-bound extension with an opaque scalar right side. This path handles bounds that the affine scalar grammar excludes. It has no stated completeness guarantee. The additive-group path also has partial coverage. Added signed integer proposals can succeed where rational averaging is unavailable. Each path that succeeds constructs a Lean-validated certificate.

Certificate identities must complete elaboration and synthesis before acceptance. Definitional vector aliases use checked normalization. Elaboration recovery does not replace a failed proof by `sorry`. The scalar backend uses exact proof construction at matching transparency.

Composed evidence. The historical full census gives the pinned source universe and its retained coverage gaps. The targeted follow-up gives a separate source subset, protocol, outcome ledger, and isolated proposals. The patch contains 50 census-generated replacements and 37 replacements composed outside the census.

The selector combines census-generated replacements with replacements composed outside the census. Replacements composed outside the census can supply explicit facts or add preparation. Artificial intelligence (AI) coding agents assisted their development. The accounting counts all added preparation.

The selector recomputes disjoint source edits and checks original declaration body observations. Final combined compilation, implementation snapshots, the frozen patch, and the new timing run form a separate chain. The historical full census tactic implementation does not supply a speed result for this chain.

Line accounting. Count nonblank, noncomment lines of proof code in the formatted source. Count preparation in each replacement. Count shared helper declarations one time.

The report counts imports and includes them in the net figure. Count only the deterministic nonoverlapping combined patch. The report also counts the tactic implementation and census infrastructure. This count does not show a net reduction in the library's total source size.

Semantic census. The census uses the sorted tracked Lean sources in `Mathlib/` at the pinned revision. The census must include tactic observations and expected-type information from term proofs. The extractor must wait for finalized elaboration data. It records original source spans, declarations, normalized goals, local facts, eligibility, attempts, and per-file completion. Assign explicit classes to macro-generated, overlapping, abandoned, or uneditable observations.

A checked replay proposal is not a verified source replacement. The full source patch must compile with all selected replacements together. The census denominator must include extraction failures and exhausted files.

The full census uses four workers. The targeted follow-up uses three. Each worker uses one Lean thread and a 600-second file limit. Each worker has a 4096 MB memory limit.

A resource-failure retry uses one worker, an 1800-second file limit, and 8192 MB. The per-goal limits are 200000 heartbeats and 2000 milliseconds. The scan makes no added preparation and does not try original single-line proofs.

An eligible observation must have an editable source span and a closed local context. Unlocated or unresolved nodes are coverage gaps even in files with complete extraction. Their counters are not counts of distinct unsupported goals.

The census has 99 known eligible observations without a recorded outcome because a file did not complete. Failed files can also have goals that extraction did not record. Counters from final file records do not count all known observations. The follow-up has 0 known eligible observations without terminal outcomes.

Table 1: Historical full census, targeted follow-up, and composed verified patch. The datasets are separate. Proof counts include preparation, helpers, and imports.

Measure	Value
Tracked files processed	8637
Files with complete extraction	8610
Files with incomplete outcomes	27
Known eligible observations	20628
Isolated checked proposals	40
Targeted follow-up files processed	885
Follow-up complete / incomplete files	875 / 10
Follow-up isolated checked proposals	158
Compiled proof spans / affected files	87 / 46
Original / replacement proof lines	582 / 226
Added helper / import lines	0 / 46
Net proof lines after helpers and imports	310
Ratio of mean compilation times	1.0724
Paired 95% interval	[1.0669, 1.0814]

Historical source format correction. An earlier patch had a proof line longer than mathlib's source-style limit. The format correction removed that file's net line saving. The historical 41-line patch then passed new source verification, calibration, and confirmatory timing. The historical timing runs do not give an estimate for the expanded patch.

Table 2: Final file outcomes and outcomes for known eligible observations. The counters use final file records and exclude files without those records. Known observations without outcomes are not tactic failures.

Recorded outcome	Count
augmented_elaboration_failed	4
complete	8610
file_timeout	1
original_elaboration_failed	5
partial_extraction	14
process_failed	3
attempt: augmented_elaboration_failed	15
attempt: context_unmatched	4
attempt: failed	2753
attempt: import_cycle	54
attempt: no_line_saving_untried	17663
attempt: success	40
Unknown due to incomplete files	99
final file record: duplicate	19598
final file record: eligible	20385
final file record: errors	54
final file record: normGoals	122441
final file record: outsideBody	448
final file record: proofNodes	5164910
final file record: synthetic	0
final file record: tacticNodes	5945347
final file record: termNodes	34310511
final file record: unlocatedNodes	12037137
final file record: unresolved	12445

Confirmatory timing. We used paired passes over exactly the same affected files. Dependencies were built before measurement. Each pass emitted fresh compiler outputs. The run alternated the two variant sequences. For original totals $O_1, \ldots, O_N$ and replacement totals $R_1, \ldots, R_N$, the primary ratio is

$$S = \frac{\frac{1}{N}\sum_t O_t}{\frac{1}{N}\sum_t R_t}.$$

We grouped the N paired passes in sequence into $K = N/b$ blocks of the calibrated even length b. The blocks have equal length and do not overlap. Each block includes the two variants and all affected files. For each replicate, the bootstrap selects K full blocks with replacement. It keeps all observations of each selected block together. The even block length gives equal counts of the two variant sequences.

The sequences are original then replacement, and replacement then original. There are K sampling units.

The method uses the bias-corrected and accelerated (BCa) probability transformation [Efron, 1987]. For acceleration, the jackknife removes one full block at a time [DiCiccio and Efron, 1996]. The implementation gives half weight to bias-correction ties. It uses linear interpolation for empirical quantiles.

A bootstrap method has model assumptions [Field and Welsh, 2007]. The interval uses a model with independent, representative blocks for the recorded timing conditions. The calibration uses pilot diagnostics to select the block length. These diagnostics do not prove the assumptions.

Timing conditions can change over time. The interval is for runs of this fixed workload on the recorded machine. It does not give a result for other hardware or mathlib revisions. The paper gives no coverage theorem for arbitrary timing dependence.

We fixed the sample count, block length, seed, bootstrap count, and stopping rule before the confirmatory run. The success criterion was a full paired 95% interval for S greater than one.

Table 3: Observed fragments and their recorded outcomes. These counts are not combined patch savings.

Fragment and outcome	Count
observed: additive_group_extension	12306
observed: core	14628
observed: outside_fragment	90124
observed: upper_bound_extension	6215
eligible: additive_group_extension	8139
eligible: core	8989
eligible: upper_bound_extension	3500
additive_group_extension: augmented_elaboration_failed	8
additive_group_extension: context_unmatched	2
additive_group_extension: failed	958
additive_group_extension: import_cycle	30
additive_group_extension: no_line_saving_untried	7094
additive_group_extension: success	15
core: failed	1077
core: import_cycle	2
core: no_line_saving_untried	7887
core: success	23
upper_bound_extension: augmented_elaboration_failed	7
upper_bound_extension: context_unmatched	2
upper_bound_extension: failed	718
upper_bound_extension: import_cycle	22
upper_bound_extension: no_line_saving_untried	2682
upper_bound_extension: success	2

Table 4: Targeted follow-up outcomes. This subset does not give a new full-universe census.

Recorded outcome	Count
file: augmented_elaboration_failed	8
file: complete	875
file: partial_extraction	2
attempt: augmented_elaboration_failed	42
attempt: context_unmatched	8
attempt: failed	6475
attempt: goal_timeout	38
attempt: import_cycle	192
attempt: no_line_saving_untried	20742
attempt: success	158
Unknown due to incomplete files	0

The block-length review used a separate 24-pair pilot. It excluded absolute times, average speed levels, confidence intervals, and significance. It selected 4 pairs for each block before the confirmatory run. There were 6 pilot blocks at this block length. These diagnostics do not prove block independence.

Table 5: Known eligible follow-up observations by original goal fragment. Isolated proposals and composed patch savings are separate.

Fragment	Count
additive_group_extension	8936
core	10362
metric_extension	4065
upper_bound_extension	4292

Table 6: Deterministic selection outcomes before source verification. The selection ledger records the causes. The verified combined patch gives the proof savings.

Selection outcome	Count
duplicate_replacement	23
import_cost_offsets_savings	29
no_line_saving	8
overlap	61
selected	87

Table 7: Implementation code has a separate cost from the proposed mathlib proof savings. These totals do not show a reduction in total repository size.

Component	Code lines
Tactic implementation	850
Census infrastructure	470

Table 8: Blinded pilot diagnostics for the frozen block-length decision. The review used serial, drift, and order metrics. These diagnostics do not prove independence.

Diagnostic	Value
Measured pilot pairs	24
Duration correlation at lag 1 (original / patch)	0.125 / 0.173
Duration correlation at lag 2 (original / patch)	0.011 / -0.269
4-pair duration correlation at lag one	0.218 / -0.260
8-pair duration correlation at lag one	-0.182 / -0.391
Half-run duration change (%)	-2.469 / -0.703
Change in mean log ratio	-0.017
AB minus BA relative duration (%)	0.861 / 0.266
AB minus BA mean log ratio	0.005
4-pair log-ratio correlations at lags 1–3	0.323 / -0.051 / -0.252
Pilot / confirmatory blocks	6 / 10

The analysis kept slower files, failed attempts, and measured passes. Timeouts or incomplete variants prevent a complete-compilation claim for those variants. The table separates setup processes from unmeasured dependency and native builds. Search-based compilation is the primary measurement. The study does not give timing results for explicit certificate replay.

The confirmatory run uses 10 blocks of 4 paired passes. It uses 10000 bootstrap samples with seed 20261020. The ratio of mean full-pass times is 1.0724. The conditional paired 95% interval is [1.0669, 1.0814].

The predeclared speed criterion is met. The analysis includes all affected files and measured passes.

Second-half mean times changed by +4.26% for original and +5.14% for replacement relative to the first half. The interval remains conditional on independent, representative timing blocks.

Table 9: Confirmatory means in seconds for each affected file. The ratio is original time divided by replacement time. The analysis includes slower files.

Affected file	Original	Patch	Ratio
`Mathlib/Analysis/Analytic/Basic.lean`	5.114	4.555	1.1228
`Mathlib/Analysis/Asymptotics/AsymptoticEquivalent.lean`	1.906	1.711	1.1143
`Mathlib/Analysis/Asymptotics/SpecificAsymptotics.lean`	1.659	1.538	1.0788
`Mathlib/Analysis/Calculus/BumpFunction/FiniteDimension.lean`	3.036	2.715	1.1181
`Mathlib/Analysis/Calculus/FDeriv/Basic.lean`	3.283	2.890	1.1362
`Mathlib/Analysis/Calculus/FDeriv/Measurable.lean`	4.400	4.185	1.0514
`Mathlib/Analysis/Complex/OpenMapping.lean`	2.170	2.014	1.0777
`Mathlib/Analysis/Convex/Uniform.lean`	1.288	1.738	0.7414
`Mathlib/Analysis/Fourier/RiemannLebesgueLemma.lean`	2.260	2.180	1.0364
`Mathlib/Analysis/LocallyConvex/Polar.lean`	1.726	1.651	1.0452
`Mathlib/Analysis/Normed/Algebra/Spectrum.lean`	4.085	3.723	1.0974
`Mathlib/Analysis/Normed/Algebra/Unitization.lean`	2.629	2.586	1.0165
`Mathlib/Analysis/Normed/Field/Approximation.lean`	1.585	1.571	1.0086
`Mathlib/Analysis/Normed/Group/ControlledClosure.lean`	1.235	1.275	0.9686
`Mathlib/Analysis/Normed/Module/MStructure.lean`	1.549	1.663	0.9315
`Mathlib/Analysis/Normed/Module/Multilinear/Basic.lean`	8.264	7.739	1.0677
`Mathlib/Analysis/Normed/Operator/Banach.lean`	3.972	3.881	1.0233
`Mathlib/Analysis/Normed/Order/Lattice.lean`	1.320	1.340	0.9851
`Mathlib/Analysis/SpecialFunctions/Complex/LogBounds.lean`	3.207	2.995	1.0709
`Mathlib/Analysis/SpecialFunctions/Elliptic/Weierstrass.lean`	9.987	9.200	1.0855
`Mathlib/Analysis/SpecialFunctions/Exp.lean`	1.949	1.678	1.1612
`Mathlib/Analysis/SpecialFunctions/FrullaniIntegral.lean`	1.978	1.831	1.0806
`Mathlib/Computability/AkraBazzi/GrowsPolynomially.lean`	3.483	3.128	1.1136
`Mathlib/MeasureTheory/Constructions/HaarToSphere.lean`	2.572	2.544	1.0110
`Mathlib/MeasureTheory/Covering/BesicovitchVectorSpace.lean`	2.594	2.452	1.0580
`Mathlib/MeasureTheory/Function/AEEqOfIntegral.lean`	1.933	1.771	1.0918
`Mathlib/MeasureTheory/Function/ConvergenceInDistribution.lean`	2.140	1.951	1.0967
`Mathlib/MeasureTheory/Function/Jacobian.lean`	4.147	3.669	1.1303
`Mathlib/MeasureTheory/Function/L1Space/Integrable.lean`	4.264	3.632	1.1741
`Mathlib/MeasureTheory/Integral/DominatedConvergence.lean`	2.912	2.511	1.1597
`Mathlib/MeasureTheory/Integral/FinMeasAdditive.lean`	3.182	2.854	1.1150
`Mathlib/MeasureTheory/Integral/PeakFunction.lean`	2.574	2.353	1.0940

Affected file	Original	Patch	Ratio
`Mathlib/MeasureTheory/Measure/IntegralCharFun.lean`	1.766	1.618	1.0918
`Mathlib/MeasureTheory/Measure/SeparableMeasure.lean`	2.237	2.191	1.0209
`Mathlib/NumberTheory/Padics/PadicNumbers.lean`	3.785	3.381	1.1195
`Mathlib/Probability/StrongLaw.lean`	3.850	3.441	1.1190
`Mathlib/Topology/Bornology/BoundedOperation.lean`	1.279	1.206	1.0609
`Mathlib/Topology/ContinuousMap/Bounded/ArzelaAscoli.lean`	1.117	1.273	0.8775
`Mathlib/Topology/ContinuousMap/Bounded/Basic.lean`	2.578	2.325	1.1087
`Mathlib/Topology/ContinuousMap/Bounded/Normed.lean`	3.469	3.188	1.0882
`Mathlib/Topology/MetricSpace/Gluing.lean`	1.804	1.744	1.0347
`Mathlib/Topology/MetricSpace/GromovHausdorff.lean`	3.251	3.012	1.0793
`Mathlib/Topology/MetricSpace/GromovHausdorffRealized.lean`	2.208	2.007	1.1002
`Mathlib/Topology/MetricSpace/Kuratowski.lean`	1.302	1.346	0.9671
`Mathlib/Topology/MetricSpace/PiNat.lean`	2.825	2.526	1.1184
`Mathlib/Topology/MetricSpace/Polish.lean`	1.313	1.552	0.8456
Total	131.188	122.333	1.0724

Table 10: Recorded setup costs outside measured compilation times. These costs do not include a full build without dependency artifacts.

Setup measurement	Seconds
Recorded setup processes	296.029
Separate tactic rebuild	Not measured
Build without dependency/native artifacts	Not measured

The variants use native plugin configurations from their recorded headers. The replacement imports can load Mathlib and tactic native libraries for unchanged tactics. The ratio includes these import configurations. It does not prove an algorithmic improvement. A separate algorithm comparison must use the same runtime and imports.

The study did not measure a full build without dependency or native-library artifacts. The numerical runtime was Python 3.15.0. The recorded runtime and numerical rules define the exact bootstrap hashes.

Recorded hardware. The hardware profile records an Apple M4 Max MacBook Pro with 36 GiB of memory. The logical processor count is 14. The recorded operating system is macOS 26.7.1.

Snapshots before and after the study show the source label `AC Power`. They show low-power mode off and sleep disabled. The battery snapshots show 80% before the study and 41% after it. The later snapshot also shows battery discharging.

Queries for thermal status failed. These snapshots do not show continuous power or thermal stability. The study did not control background activity.

D Language, reproducibility, and research scope

The prose uses a documented review based on ASD-STE100 [Aerospace, Security and Defence Industries Association of Europe, 2025]. Mathematical terms, formal notation, Lean identifiers, source quotations, and bibliographic titles have declared treatment. The audit lists technical terminology, exceptions, automated checks, and the limits of those checks. This is not an ASD certification. The style uses a short mathematical-reasoning workshop format. There is no submission or acceptance claim.

The source package includes the unchanged official NeurIPS 2026 style, a bibliography, generation code, and a reproducible build command. It imports examples from checked Lean files and tables from archived JSON data. The build rejects stale generated examples, missing glyphs, unresolved references, and a main body longer than four pages. The build renders the PDF for visual review. A rebuild can give a different PDF byte sequence on different TeX installations.

E Implementation excerpts and guide

The three excerpts show the certificate lemmas, reference search, and proof construction used by the measured tactic. The build imports their text from the archived source and checks its hashes. Line numbers are from the original files. The full source snapshot includes all five project modules and the omitted helpers. The source package also includes these complete files and their hash manifest, `generated/implementation.json`.

E.1 Small norm certificates

In `NormArith/Certificate.lean`, E has a `SeminormedAddCommGroup E` instance. The excerpt adds the real normed-space structure. The lemma `bound_smul` scales a supplied bound by the absolute value of its coefficient. The lemma `bound_add` uses the triangle inequality to combine two bounds. Their composition proves the two-term certificate below. Lean checks these lemmas and each later application.

```
variable [NormedSpace ℝ E]

/-- Scale an upper bound by the absolute value of a real coefficient. -/
theorem bound_smul (c : ℝ) {x : E} {b : ℝ}
    (hx : ‖x‖ ≤ b) : ‖c • x‖ ≤ |c| * b := by
  rw [norm_smul, Real.norm_eq_abs]
  exact mul_le_mul_of_nonneg_left hx (abs_nonneg c)

/-- A two-term signed linear-combination certificate. -/
theorem norm_combo_le (c d : ℝ) {x y : E} {a b : ℝ}
    (hx : ‖x‖ ≤ a) (hy : ‖y‖ ≤ b) :
    ‖c • x + d • y‖ ≤ |c| * a + |d| * b :=
  bound_add (bound_smul c hx) (bound_smul d hy)
```

E.2 Exact rational search

The function below is from `NormArith/Search.lean`. Each column records a norm argument in shared formal atom coordinates. The helper `solve` uses exact Gauss–Jordan elimination over rational numbers. The reference search enumerates independent supports through the column rank and checks each proposed coefficient array. This representation does not imply linear independence of the concrete vectors.

The default limits are 10,000 tested subsets and 10,000 returned proposals per target. The result records completion, resource exhaustion, or invalid dimensions. A completed enumeration gives the proposals for the finite reduction. The frontend must prove their vector identities. This search is not a complete procedure for arbitrary additive groups.

```
def referenceDecompositions (sources : Array (Array Rat)) (target : Array Rat)
    (config : ReferenceConfig := {}) : ReferenceResult := Id.run do
  if sources.any (fun v => v.size != target.size) then
    return { status := .invalidDimensions }
  let zero : Array Rat := Array.replicate sources.size 0
  let whole := solve sources target (List.range sources.size).toArray
  let rank := whole.rank
  if whole.solution?.isNone then
    return { rank }
  if target.all (· == 0) then
    if config.maxResults == 0 then
      return { rank, status := .exhausted .results }
    return { coefficients := #[zero], rank }
  let mut result : ReferenceResult := { rank }
  for support in [1:rank + 1] do
    let mut selected? := some (List.range support).toArray
    while let some selected := selected? do
      if result.attempts >= config.maxSubsets then
        return { result with status := .exhausted .subsets }
      result := { result with attempts := result.attempts + 1 }
      let solved := solve sources target selected
      if solved.rank == selected.size then
        if let some solution := solved.solution? then
```

```
          let mut coefficients := zero
          for index in [:selected.size] do
            coefficients := coefficients.set! selected[index]!
            solution[index]!
          if isDecomposition sources target coefficients &&
          !result.coefficients.contains coefficients then
            if result.coefficients.size >= config.maxResults then
              return { result with status := .exhausted .results }
            result := { result with coefficients := result.coefficients.push
            coefficients }
        selected? := nextSubset? sources.size selected
    return result
```

E.3 From coefficients to a checked proof

The function `addConstraint` is from `NormArith/Tactic.lean`. It constructs a vector combination and a proof of its norm bound. The helper `scaleSource` selects a lemma for the coefficient. Repeated applications of `bound_add` combine the bounds. Reflexivity, `abel!`, or `module` proves the vector identity. The lemma `bound_of_eq` then gives the target bound.

The code checks term elaboration and simplifies coefficient arithmetic. The caller can then use linear arithmetic to close the goal. A failed attempt restores the saved state. If no proof closes the goal, exhaustion of the finite search reports unknown. The search proposes coefficients, and Lean checks the resulting proof.

```
private def addConstraint (sources : Array Source) (target : Expr)
    (coefficients : Array Rat) : TacticM Expr := Term.withoutErrToSorry do
  let space ← Term.exprToSyntax (← inferType target)
  let mut combination? : Option (TSyntax `term × TSyntax `term) := none
  for i in [:sources.size] do
    if coefficients[i]! == 0 then continue
    let v ← Term.exprToSyntax sources[i]!.vector
    let h ← Term.exprToSyntax sources[i]!.proof
    let (multiple, bound) ← scaleSource coefficients[i]! v h
    combination? ← match combination? with
      | none => pure (some (multiple, bound))
      | some (v, h) => do
        pure (some (← `($multiple + $v), ← `(NormArith.bound_add $bound $h)))
  let (combination, proof) ← match combination? with
    | some pair => pure pair
    | none => do pure (← `((0 : $space)), ← `(NormArith.bound_zero (E :=
    $space)))
  let combinationExpr ← Term.elabTermAndSynthesize combination (some (←
  inferType target))
  let identity ← if ← isDefEq target combinationExpr then mkEqRefl target else
  do
    let tactic ← if coefficients.all (·.den == 1) then
      `(by first | (abel!; done) | module) else `(by module)
    Term.elabTermAndSynthesize tactic (some (← mkEq target combinationExpr))
  let bound ← Term.elabTermAndSynthesize proof none
  let certificate ← mkAppM ``bound_of_eq #[identity, bound]
  let userName ← mkFreshUserName `norm_arith_constraint
  let name := mkIdent userName
  let (fvar, next) ← (← getMainGoal).note userName certificate
  replaceMainGoal [next]
  if coefficients.all (fun c => c == 0 || c == 1 || c == -1) then
    return mkFVar fvar
  unless coefficients.all (fun c => c == 0 || c == 1 || c == -1) do
    evalTactic (← `(tactic| norm_num at $name:ident))
  if (← getGoals).isEmpty then return ← mkConst' ``True.intro
  withMainContext ◁ Term.elabTermAndSynthesize name none
```

The frontend tries scalar arithmetic and short decompositions before the reference search. Other paths normalize facts, specialize quantified bounds with fixed limits, and add metric constraints for at most eight points. The full source includes these paths and the public tactic syntax. Appendix B gives examples and extension limits. The mathematical completeness theorem does not prove unrestricted completeness for the bounded program.